

FINAL YEAR PROJECT REPORT

KrishiGyaan

AI-Powered Agriculture Support Platform for Indian Farmers

Programme	Bachelor of Computer Applications (BCA)
Document	Phase 1 - Analysis, Requirements, Design and Testing
Submitted by	_____
Enrollment No.	_____
Project Guide	_____
Academic Session	_____
Institution	_____

Submitted in partial fulfilment of the requirements for the award of the Bachelor of Computer Applications degree.

Project Certificate

This is to certify that the project report entitled **KrishiGyaan - AI-Powered Agriculture Support Platform for Indian Farmers** is an original final-year project prepared in partial fulfilment of the requirements for the Bachelor of Computer Applications degree.

The work described in this Phase 1 volume covers requirement analysis, software design, database design, system diagrams, validation, deployment and user-interface documentation. Source-code listings are intentionally reserved for Phase 2.

Student signature	_____
Guide signature	_____
Date	_____
Place	_____

Self-Certificate

I declare that this report and the associated KrishiGyaan application represent authentic project work. Material taken from technical documentation and third-party services has been acknowledged in the bibliography. The project has not been submitted earlier for another degree or diploma.

I understand that the application depends on external services and that all API keys, database credentials and administrator secrets must remain outside the source repository.

Student name	_____
Enrollment number	_____
Signature	_____
Date	_____

Certificate from Project Guide

This is to certify that the project titled **KrishiGyaan** was carried out under my guidance. The report documents the current implementation honestly, including its design decisions, external dependencies, security controls, limitations and future scope.

Guide name	_____
Designation	_____
Department / Centre	_____
Signature and date	_____

Acknowledgement

I express sincere gratitude to the project guide, faculty members, institution, friends and family members whose guidance and feedback supported the completion of this final-year project.

I also acknowledge the maintainers of Node.js, MongoDB, Vercel, Groq, Open-Meteo, Cloudinary, Crop Kindwise and Plant.id. Their platforms and documentation made it possible to construct and evaluate an integrated agricultural assistance system.

- Guidance on software engineering and documentation
- Feedback on usability and responsive presentation
- Access to development, database and deployment tools
- Encouragement during implementation and testing

Abstract

KrishiGyaan uses a static HTML, CSS and vanilla JavaScript client with Node.js serverless functions. MongoDB Atlas stores farmer profiles and the latest dashboard state. Groq produces constrained agricultural language output, Open-Meteo supplies forecasts, Crop Kindwise and Plant.id analyse images, and Cloudinary stores uploaded photographs.

The central problem is fragmentation: weather, crop diagnosis, schemes, soil advice and expert questions are normally handled by separate channels. KrishiGyaan presents these services in one farmer profile-aware dashboard and supports multilingual text, speech input and browser dictation.

The application is deployed as a Vercel-compatible web project. It preserves the latest farmer analysis in MongoDB, stores submitted images in Cloudinary and provides an administrator data centre. The result is a practical academic prototype demonstrating database, web, API, AI and software-engineering concepts in one system.

Project Report Summary

Subject / title	KrishiGyaan - AI-Powered Agriculture Support Platform
Project category	Web application, DBMS and AI-assisted expert system
Front end	HTML5, CSS3 and vanilla JavaScript
Back end	Node.js serverless API and local HTTP server
Database	MongoDB Atlas
Deployment	Vercel
External services	Groq, Open-Meteo, Cloudinary, Crop Kindwise and Plant.id
Document scope	Phase 1: analysis, design, testing and UI documentation

Table of Contents - Part I

Introduction	12
Objectives and scope	15
System analysis and feasibility	18
Planning and SDLC model	24
Software Requirements Specification	29
Architecture and diagrams	40
System and module design	51

Table of Contents - Part II

Database design and data dictionary	58
Security and error handling	64
Testing and validation	67
User interface documentation	73
Deployment, limitations and future scope	85
Bibliography	90

1. Introduction

KrishiGyaan uses a static HTML, CSS and vanilla JavaScript client with Node.js serverless functions. MongoDB Atlas stores farmer profiles and the latest dashboard state. Groq produces constrained agricultural language output, Open-Meteo supplies forecasts, Crop Kindwise and Plant.id analyse images, and Cloudinary stores uploaded photographs.

A farmer creates one profile containing identity, location, land, crop, season, irrigation and benefit-access details. That profile provides context to weather, scheme, soil and AI services so repeated data entry is reduced and advice remains consistent.

The project combines a public information site, farmer account workflow, protected service dashboard and administrator data centre. It is a standalone academic application rather than a team or event landing page.

1.1 Background and Domain Context

Indian agriculture decisions are highly dependent on place, season, rainfall, crop stage, soil condition and access to public support. Digital information exists, but it is commonly scattered across portals and technical interfaces.

A profile-aware application can reduce repeated searching and present the same data in an actionable order: understand the farmer, check weather, assess crop or soil signals, identify support and ask follow-up questions.

1.2 Problem Definition

Farmers may receive contradictory or delayed advice because weather, schemes, disease information and general guidance are consulted independently. Language barriers, low connectivity and complex forms further reduce access.

The project problem is to create one responsive system that securely stores a farmer profile and connects it to understandable, current and recoverable agricultural guidance without exposing provider credentials.

2. Project Objectives

The primary objective is to develop an integrated, profile-aware agricultural assistance platform suitable for a final-year BCA project.

- Create secure farmer registration and authentication.
- Display weather trends as field actions rather than raw values.
- Support crop, plant and soil assessment workflows.
- Match government schemes and prepare language-specific drafts.
- Preserve previous analysis after refresh.
- Provide administrator CRUD and image visibility.

2.1 Scope

The implemented scope covers public pages, farmer authentication, dashboard services, MongoDB persistence, Cloudinary images, external agricultural APIs and administrative control.

The system does not replace an agriculture officer, laboratory test, bank decision or official scheme portal. Recommendations are decision support and must be checked against local conditions.

2.2 Project Category

KrishiGyaan is a database-backed web application with AI-assisted expert-system characteristics. It also demonstrates API integration, cloud deployment, responsive interface design and multilingual accessibility.

The DBMS component manages persistent profiles and analyses; the expert-assistance component transforms user context and external observations into structured guidance.

3. Existing System Analysis

The existing information environment is fragmented across government portals, weather applications, helplines, banks, local experts and commercial diagnosis services. Each source usually requires separate navigation and repeated context.

Common weaknesses include technical terminology, inconsistent language support, no shared farmer profile, no unified history and limited connection between a forecast and a farm task.

3.1 Proposed System

A farmer creates one profile containing identity, location, land, crop, season, irrigation and benefit-access details. That profile provides context to weather, scheme, soil and AI services so repeated data entry is reduced and advice remains consistent.

KrishiGyaan uses a static HTML, CSS and vanilla JavaScript client with Node.js serverless functions. MongoDB Atlas stores farmer profiles and the latest dashboard state. Groq produces constrained agricultural language output, Open-Meteo supplies forecasts, Crop Kindwise and Plant.id analyse images, and Cloudinary stores uploaded photographs.

The proposed system uses a simple navigation flow and stores only the latest dashboard analysis required for continuity. External responses are converted into readable cards, graphs, plans and draft documents.

3.2 Preliminary Investigation

The investigation identified seven recurring needs: trusted account access, field planning, weather interpretation, disease detection, scheme support, soil guidance and open-ended questions.

The repository and rendered screens were examined to map each need to a page, API route, storage collection and external provider.

3.3 Technical Feasibility

The chosen stack is technically feasible because the browser requires no framework runtime, Node.js functions fit Vercel deployment, and MongoDB Atlas supplies managed document storage.

All provider integrations use HTTP and JSON. The implementation already passes syntax checks and can run locally using the same environment contract used in production.

3.4 Operational Feasibility

The farmer workflow uses familiar forms, buttons and cards. Disabled submit buttons prevent empty AI requests, language selection controls output, and prior results remain available after refresh.

The administrator can locate a farmer, review saved output and deliberately enter edit mode. Destructive actions request confirmation.

3.5 Economic and Schedule Feasibility

The prototype uses free or entry-level service tiers and a small runtime dependency set. Major costs are provider usage, cloud bandwidth, maintenance and any future SMS or expert-verification service.

An incremental academic schedule is practical: requirements, interface prototype, persistence, external integrations, testing, deployment and documentation.

3.6 Cost-Benefit Analysis

Development cost is limited by the use of open web standards and managed services. Operational cost grows with AI tokens, image analysis requests, image storage and database traffic.

Benefits include reduced information switching, reusable profile context, earlier issue recognition, clearer scheme preparation and easier demonstration of integrated software engineering.

4. Planning and Scheduling

Work was divided into requirements, data modelling, authentication, dashboard modules, API integration, persistence, administration, responsive refinement, testing and documentation.

Risk-heavy work such as external API integration and multilingual generation was scheduled before final interface polishing so provider constraints could influence validation and fallback behaviour.

4.1 Indicative Gantt Plan

Weeks 1-2	Problem study, objectives and SRS
Weeks 3-4	Wireframes, data model and authentication
Weeks 5-7	Dashboard and external APIs
Weeks 8-9	Persistence, Cloudinary and admin
Weeks 10-11	Testing and responsive correction
Weeks 12-13	Deployment and Phase 1 documentation

4.2 Selected Software Model - Waterfall

The report uses the Waterfall model as the principal academic model because the BCA deliverables require clear, sequential evidence: requirements, design, implementation, testing, deployment and documentation.

Small corrective iterations occurred inside stages, but approval moves forward only after the current artefact is checked. This gives traceability between SRS items, modules and test cases.

- Requirements analysis
- System and database design
- Implementation
- Integration and testing
- Deployment
- Maintenance and enhancement

4.3 SDLC Application

During planning, user needs were converted into requirements. Design produced page flow, API contracts and MongoDB collections. Implementation created static pages and serverless handlers. Testing checked syntax, routes, service fallbacks and responsive views.

Deployment uses Vercel rewrites and environment variables. Maintenance includes provider-model changes, scheme updates, secret rotation and future usability research.

4.4 Risk Management

Primary risks are provider rate limits, missing deployment variables, invalid images, stale government information, network loss and accidental administrative deletion.

The design favours a small dependency surface, explicit validation, server-side secrets, stable JSON contracts and recoverable user state. Every external service is treated as fallible; the interface therefore explains missing configuration, unavailable providers and saved-result fallback behaviour.

- Use fallback AI credentials and concise prompts.
- Keep prior analysis in MongoDB.
- Validate image size and MIME type.
- Keep secrets outside Git.
- Require confirmation for deletes.

5. SRS - Purpose and Product Perspective

This SRS defines observable behaviour for farmers and administrators. KrishiGyaan operates as a browser client backed by stateless Node.js request handlers and persistent cloud services.

The product boundary includes presentation, validation, API orchestration and stored records. Official eligibility decisions, medical-grade plant diagnosis and guaranteed agronomic outcomes remain outside the boundary.

5.1 User Classes

The primary user is an Indian farmer using a phone or desktop browser. The secondary user is an authorised administrator responsible for records and support. External actors are AI, weather, image-analysis, image-storage and database services.

- Farmer: registers, logs in, runs tools and reads saved results.
- Administrator: manages users, analysis and uploads.
- External provider: returns data or a controlled error.

5.2 Functional Requirements - Registration

The system shall validate full name, Indian mobile number, age, location, land, crop, benefit-access and password fields. It shall reject duplicate mobile numbers and store a salted password hash rather than plaintext.

A successful registration shall return a public profile without password material.

5.3 Functional Requirements - Authentication

The system shall authenticate mobile number and password, update last-login time and place the public profile in browser storage for dashboard access. Invalid credentials shall return a clear error.

The prototype forgot-password route performs account lookup only and must not be treated as a production recovery mechanism.

5.4 Functional Requirements - Weather

The dashboard shall request location permission, resolve an understandable location and obtain ten-day weather values. It shall display temperature, rainfall and wind together and produce field actions aligned with the registered crop.

Where longer-term data is unavailable, the system shall label the shorter forecast fallback rather than present it as thirty-day certainty.

5.5 Functional Requirements - Health and Soil

The farmer shall choose crop or plant mode, upload an image and receive structured provider results. The uploaded image shall be compressed, stored in Cloudinary and recorded in MongoDB when configured.

Soil guidance shall combine profile data, optional photograph signals and weather context. Results are advisory rather than laboratory measurements.

5.6 Functional Requirements - Schemes

The dashboard shall compare profile attributes with a maintained scheme catalogue, show likely matches and link to relevant official destinations.

The scheme assistant shall accept one or more characters, answer eligibility questions and generate bounded application drafts only in the selected supported language.

5.7 Functional Requirements - KrishiBaba

The assistant shall accept farming questions about crops, pests, fertilizer, irrigation, schemes, weather and market readiness. Empty prompts shall not be submitted.

AI output shall follow the selected website language, remain concise enough for provider limits and show a friendly retry message during rate limiting.

5.8 Functional Requirements - Persistence

The system shall store one latest dashboard-analysis record per farmer. After refresh, it shall restore saved sections and metrics until the farmer runs a tool again.

New successful output shall update both the visible dashboard and MongoDB. Offline snapshots may provide an additional local recovery layer.

5.9 Functional Requirements - Administration

The administrator uses a separate signed session token to inspect farmer records, saved analyses and Cloudinary uploads. Create, read, update and delete actions are exposed through one authenticated API action contract, with confirmation before destructive operations.

Profile fields and saved content shall be read-only until Edit is selected. Image deletion shall attempt removal from Cloudinary and MongoDB. Farmer deletion shall cascade to related analysis and upload records.

5.10 Non-Functional Requirements

The design favours a small dependency surface, explicit validation, server-side secrets, stable JSON contracts and recoverable user state. Every external service is treated as fallible; the interface therefore explains missing configuration, unavailable providers and saved-result fallback behaviour.

- Usability: readable mobile-first controls and clear empty states.
- Performance: static assets and bounded request payloads.
- Security: server-only secrets, hashed passwords and signed admin tokens.
- Reliability: saved-result recovery and explicit provider errors.
- Maintainability: small modules and stable environment names.

5.11 External Interface Requirements

Browser interfaces use semantic HTML controls and JSON fetch requests. Serverless endpoints accept POST and OPTIONS. MongoDB uses document collections, while external providers use HTTPS.

Cloudinary receives signed multipart uploads; Groq and health providers receive server-side API keys; Open-Meteo is accessed without exposing private credentials.

5.12 Constraints and Assumptions

The system assumes internet access for live providers, valid configured credentials and a supported modern browser. AI and diagnosis quotas may vary by provider plan.

Location permission may be denied, forecast data may be incomplete, and official scheme rules may change after documentation.

5.13 Acceptance Criteria

Acceptance requires successful page loading, farmer registration and login, persisted dashboard restoration, readable weather presentation, controlled image analysis, scheme assistance, disabled empty submissions and authenticated admin CRUD.

No API secret may be present in frontend source or committed environment files.

6. Logical Architecture

KrishiGyaan uses a static HTML, CSS and vanilla JavaScript client with Node.js serverless functions. MongoDB Atlas stores farmer profiles and the latest dashboard state. Groq produces constrained agricultural language output, Open-Meteo supplies forecasts, Crop Kindwise and Plant.id analyse images, and Cloudinary stores uploaded photographs.

The browser owns presentation and lightweight validation. Vercel routes public API paths to isolated handlers. MongoDB owns durable records, while external providers remain replaceable behind server-side calls.

6.1 Structure Chart

The top-level system divides into Public UI, Farmer Authentication, Farmer Dashboard, Persistence Services, External Integrations and Administration.

Shared frontend code handles language, voice, connectivity and AI requests; page-specific scripts handle public forms, dashboard tools and admin interactions.

6.2 Context-Level DFD

At context level, the Farmer sends profile data, questions, location and images to KrishiGyaan and receives guidance, forecasts, matches and drafts. The Administrator sends authenticated management commands and receives records.

External services exchange forecast, AI, diagnosis, storage and database data with the system boundary.

6.3 Level-1 DFD

Level 1 decomposes the platform into account management, dashboard analysis, external advisory, image management and administration. Farmer identity connects each persistent record.

Data stores are Farmers, Dashboard Analyses and Dashboard Uploads. External calls do not write directly to MongoDB; server-side handlers validate and control storage.

6.4 Level-2 DFD - Authentication

Registration validates input, normalises mobile numbers, checks uniqueness, hashes the password and stores the profile. Login retrieves by mobile, recomputes the PBKDF2 hash and returns a public profile when equal.

Password salt and hash never appear in the response projection.

6.5 Level-2 DFD - Dashboard Analysis

A dashboard action combines profile context and user input, calls the required provider, renders output and schedules persistence. The save endpoint sanitises accepted section keys and upserts the latest document.

A load action resolves the farmer and returns the stored analysis without internal identity fields.

6.6 Entity-Relationship Model

Farmer is the parent entity. A farmer has zero or one current DashboardAnalysis and zero or many DashboardUploads. MongoDB stores farmer identifiers as strings in related collections while the Farmers primary identifier remains an ObjectId.

DashboardUploads also carry Cloudinary public identifiers, URLs and image metadata.

6.7 ER Cardinality and Integrity

The mobile field is unique for Farmers. DashboardAnalyses uses a unique farmerId index, enforcing one latest analysis. DashboardUploads uses a unique publicId and a compound farmerId/createdAt index.

Application logic cascades related record deletion because MongoDB document collections do not enforce foreign keys.

6.8 Use-Case Model

Farmer use cases include register, login, view restored dashboard, check weather, analyse images, match schemes, generate drafts, evaluate soil and ask KrishiBaba.

Administrator use cases include login, search, view, create, edit and delete farmer-associated data.

6.9 Activity Flow

The main activity begins with account access. After authentication, the farmer selects a dashboard service. Input is validated, the external or internal analysis runs, output is shown, and successful state is persisted.

Failure branches preserve previous results where possible and provide a retry or configuration message.

6.10 Sequence Model

For an AI request, the dashboard builds a bounded prompt, posts to /api/ai, and the handler calls the primary Groq key. On failure it calls the fallback key. A successful response is language-checked, rendered and saved.

For an upload, the browser compresses the image, the server signs the Cloudinary request, then stores returned metadata in MongoDB.

6.11 Deployment Model

Static files and Node.js serverless functions are deployed to Vercel. MongoDB Atlas, Cloudinary, Groq, Open-Meteo, Crop Kindwise and Plant.id remain managed internet services.

Local development uses backend/server.js to serve the same frontend and emulate API routes on 127.0.0.1.

7. Module Overview

The system contains public presentation, farmer authentication, shared language and voice runtime, dashboard persistence, weather advisory, health detection, schemes, soil, AI chat, cloud uploads and administration.

Each module has a clear input, process, output and failure state.

7.1 Public and Authentication Modules

The landing page explains the problem and feature set. Registration collects detailed profile context across four steps. Login restores the profile and preferred language.

Client validation improves feedback, while server validation remains authoritative.

7.2 Weather and Advisory Modules

Open-Meteo daily arrays are converted into day objects, summary metrics and a horizontally readable forecast. Farm actions depend on rain, wind, temperature and the registered crop.

Long-term guidance is labelled as forecast direction rather than certainty.

7.3 Health and Soil Modules

Health mode selects Crop Kindwise or Plant.id without changing the public endpoint contract. Provider payloads are transformed into readable results and optional low-cost AI guidance.

Soil analysis uses profile context and photo-derived hints to create a score and improvement path.

7.4 Scheme and AI Modules

A local scheme catalogue performs transparent profile matching. Groq handles follow-up questions and draft generation under strict language and length prompts.

Application validation rejects mixed-language drafts instead of displaying an unsuitable document.

7.5 Administration and Cloud Modules

The administrator uses a separate signed session token to inspect farmer records, saved analyses and Cloudinary uploads. Create, read, update and delete actions are exposed through one authenticated API action contract, with confirmation before destructive operations.

The upload API verifies farmer identity, validates JPEG/PNG/WebP data, creates a signed Cloudinary upload and stores only returned metadata. It supports either CLOUDINARY_URL or the three separate Cloudinary variables.

8. Database Overview

MongoDB was chosen because farmer profiles and dashboard sections are naturally represented as documents. It permits additive fields while indexes preserve important uniqueness and lookup rules.

Dates are stored as Date values and returned as ISO strings where required by the frontend.

8.1 Data Dictionary - Farmers

_id	ObjectId; primary identifier
mobile	String; unique normalised 10-digit number
passwordSalt	String; random PBKDF2 salt
passwordHash	String; PBKDF2-SHA512 output
fullName/state/district/village	Profile identity and location
landSize/soilType/irrigation	Farm context
primaryCrop/season/sowingDate	Crop context
bank/pmkisan/aadhaar	Benefit access context
createdAt/updatedAt/lastLoginAt	Audit timestamps

8.2 Data Dictionary - Dashboard Analyses

farmerId	String; unique reference to farmer
farmerMobile	String; lookup context
sections	Object of approved rendered-result strings
signals.profileScore	Number
signals.schemeMatches	Number
signals.weather	Object or null
signals.diseaseScore	Number or null
signals.soilScore	Number or null
createdAt/updatedAt	Audit timestamps

8.3 Data Dictionary - Dashboard Uploads

farmerId/farmerMobile	Owner reference
kind	crop, plant or soil
originalName	Sanitised client file name
publicId	Unique Cloudinary public identifier
secureUrl	HTTPS Cloudinary delivery URL
assetId/format	Provider metadata
width/height/bytes	Numeric image metadata
createdAt	Upload timestamp

8.4 API Contract Summary

/api/auth/register	Create farmer; 201 on success
/api/auth/login	Authenticate farmer; 200 on success
/api/dashboard-analysis	load/save latest analysis
/api/dashboard-upload	Signed cloud image upload
/api/ai	Primary/fallback Groq response
/api/crop-health	Crop Kindwise proxy
/api/plant-health	Plant.id proxy
/api/admin	Token-protected CRUD actions

9. Security Design

Passwords use PBKDF2-SHA512 with 120,000 iterations and random salts. API keys and database credentials are read only from server environment variables. Admin tokens are HMAC signed and expire after eight hours.

Rendered saved HTML is sanitised in the admin page, Cloudinary links are allow-listed, request methods are constrained, and public profile responses exclude password fields.

9.1 Error Handling and Recovery

Handlers return JSON errors with appropriate status codes. The frontend disables empty submissions, catches provider errors and avoids replacing saved analysis with an unsuccessful result.

Cloud configuration errors identify the required Vercel variable names without revealing values. Connectivity recovery and service-worker caches improve resilience for static assets.

10. Testing Strategy

Testing combines syntax validation, route smoke tests, handler equivalence tests, real MongoDB CRUD, real Cloudinary upload/delete verification and visual browser inspection.

The npm run check command validates every JavaScript file used by local and serverless execution.

10.1 Authentication Test Cases

Valid registration	201 and public profile
Duplicate mobile	409 conflict
Invalid mobile	400 validation error
Weak password	400 validation error
Valid login	200 and profile
Wrong password	401 error
Protected dashboard without profile	Redirect to login

10.2 Dashboard Test Cases

Empty AI question	Send remains disabled
One character question	Send becomes enabled
Weather permission accepted	Forecast cards render
Long forecast absent	Labelled short fallback
Dashboard refresh	Stored analysis restored
Invalid image	Readable validation error
Provider rate limit	Friendly retry; saved data remains

10.3 Admin and Integration Tests

Admin login	Signed token returned
Unauthorised action	401 response
Create/update farmer	MongoDB record changes
Update analysis	Approved section values saved
Cloud upload	Cloudinary URL stored
Delete upload	Cloudinary and Mongo record removed
Delete farmer	Related data cascades

10.4 Requirements Traceability

Every major SRS group maps to an interface, handler and test group: authentication to auth endpoints, persistence to dashboard-analysis, health to health.js, cloud images to dashboard-upload, and CRUD to admin.js.

Traceability reduces the risk that a visually complete screen lacks server behaviour or that an endpoint lacks a user-facing path.

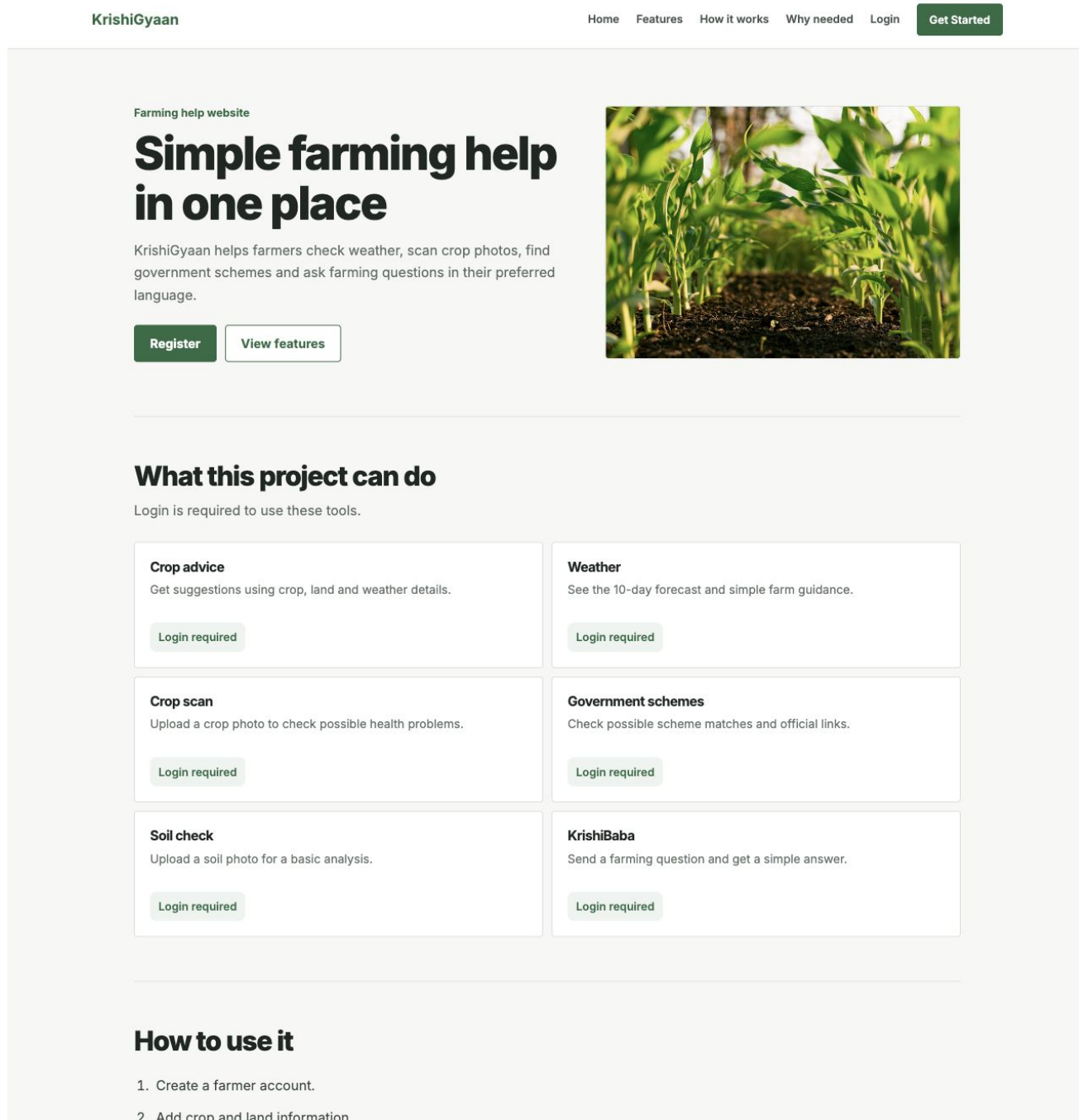
11. User Interface Design

The UI uses white surfaces, restrained green accents, clear borders and minimal motion. Navigation labels avoid decorative icons in the principal bars. Cards are used only to group related data.

Responsive layouts collapse grids without changing feature order, and form actions remain reachable on narrow phone screens.

11.1 Landing Page - View 1 of 2

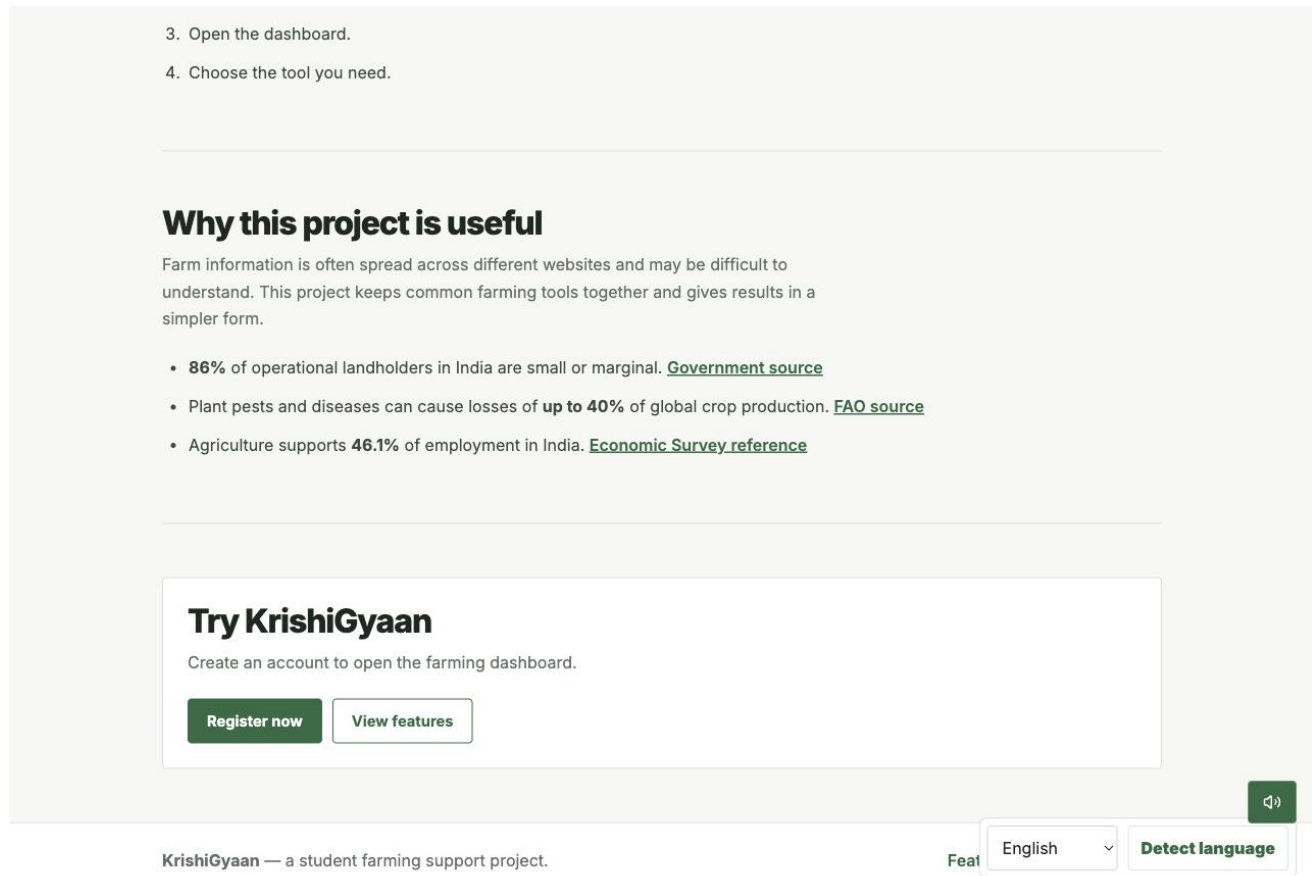
This rendered view documents the implemented interface in reading order. Controls, headings, empty states and data regions are shown as they appear in the running application.



11.1 Landing Page, segment 1 of 2.

11.1 Landing Page - View 2 of 2

This rendered view documents the implemented interface in reading order. Controls, headings, empty states and data regions are shown as they appear in the running application.



11.1 Landing Page, segment 2 of 2.

11.2 Farmer Login - View 1 of 1

This rendered view documents the implemented interface in reading order. Controls, headings, empty states and data regions are shown as they appear in the running application.

KrishiGyaan

HomeFeaturesSolutionsAboutContactLoginGet Started

SECURE FARMER ACCESS

Login to unlock every KrishiGyaan feature.

Crop health scans, weather advisory, schemes, AI chat, soil reports, and voice support are available inside the dashboard after registration and login.

 Crop scanner

 Weather advisory

 Scheme support

Farmer login

Mobile Number

9876543210

Password

Enter password

Preferred Language

English

Login to Dashboard

Forgot password?

New farmer? [Register here](#)

English

Detect language

11.2 Farmer Login, segment 1 of 1.

11.3 Farmer Registration - View 1 of 1

This rendered view documents the implemented interface in reading order. Controls, headings, empty states and data regions are shown as they appear in the running application.

KrishiGyaan[Home](#)[Features](#)[Solutions](#)[About](#)[Contact](#)[Login](#)[Get Started](#)

ADVANCED FARMER PROFILE

Register once. Get guidance for your land, crops, and schemes.

1 Personal

2 Farm

3 Crop

4 Access

Personal details

Full Name

Enter farmer name

Mobile Number

9876543210

Age

42

Gender

Select gender

State

Bihar

District

Patna

Village

Village name

Preferred Language

English

← Back

Next →

Already registered? [Login here](#)

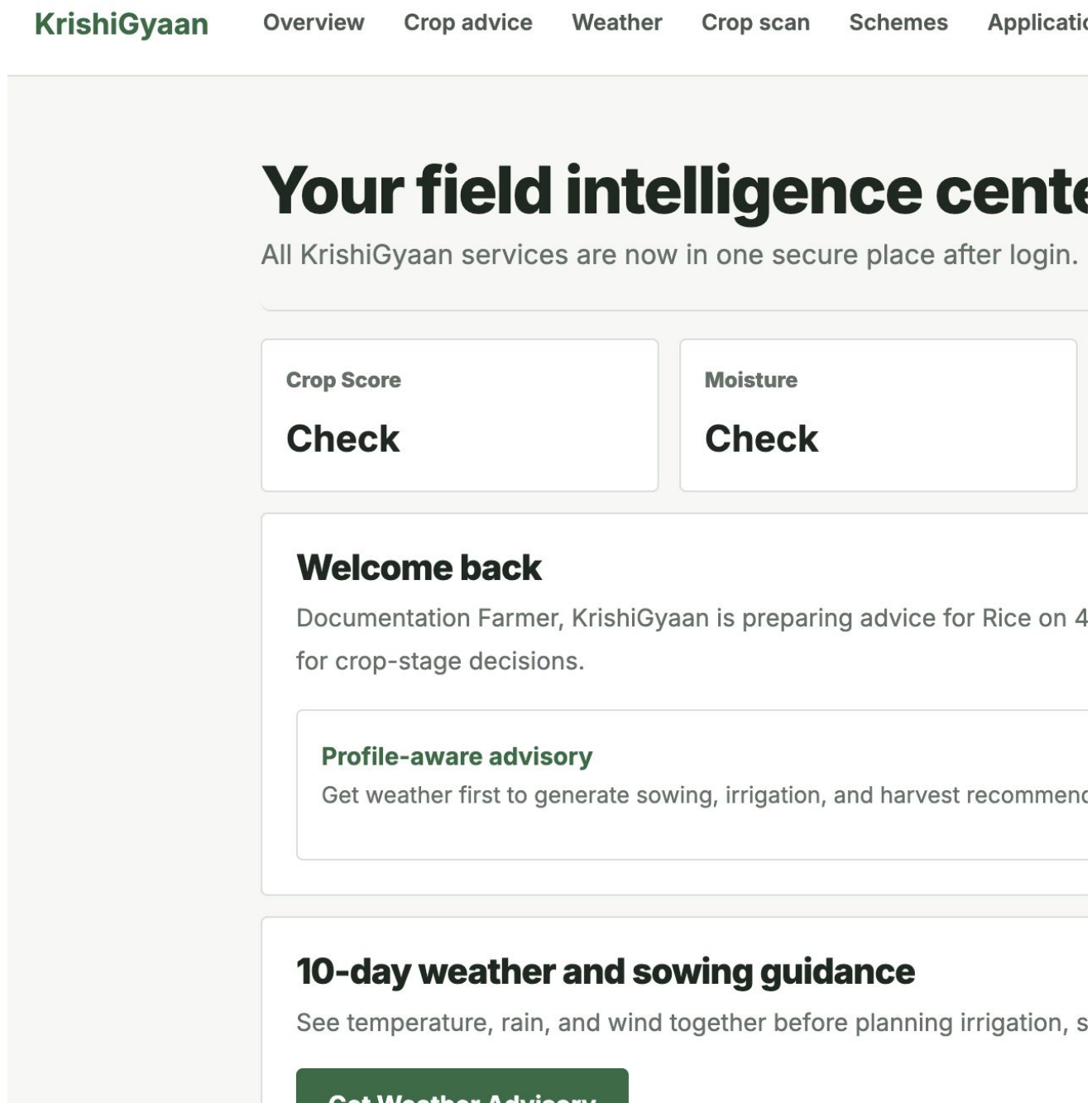
English

Detect language

11.3 Farmer Registration, segment 1 of 1.

11.4 Farmer Dashboard - View 1 of 4

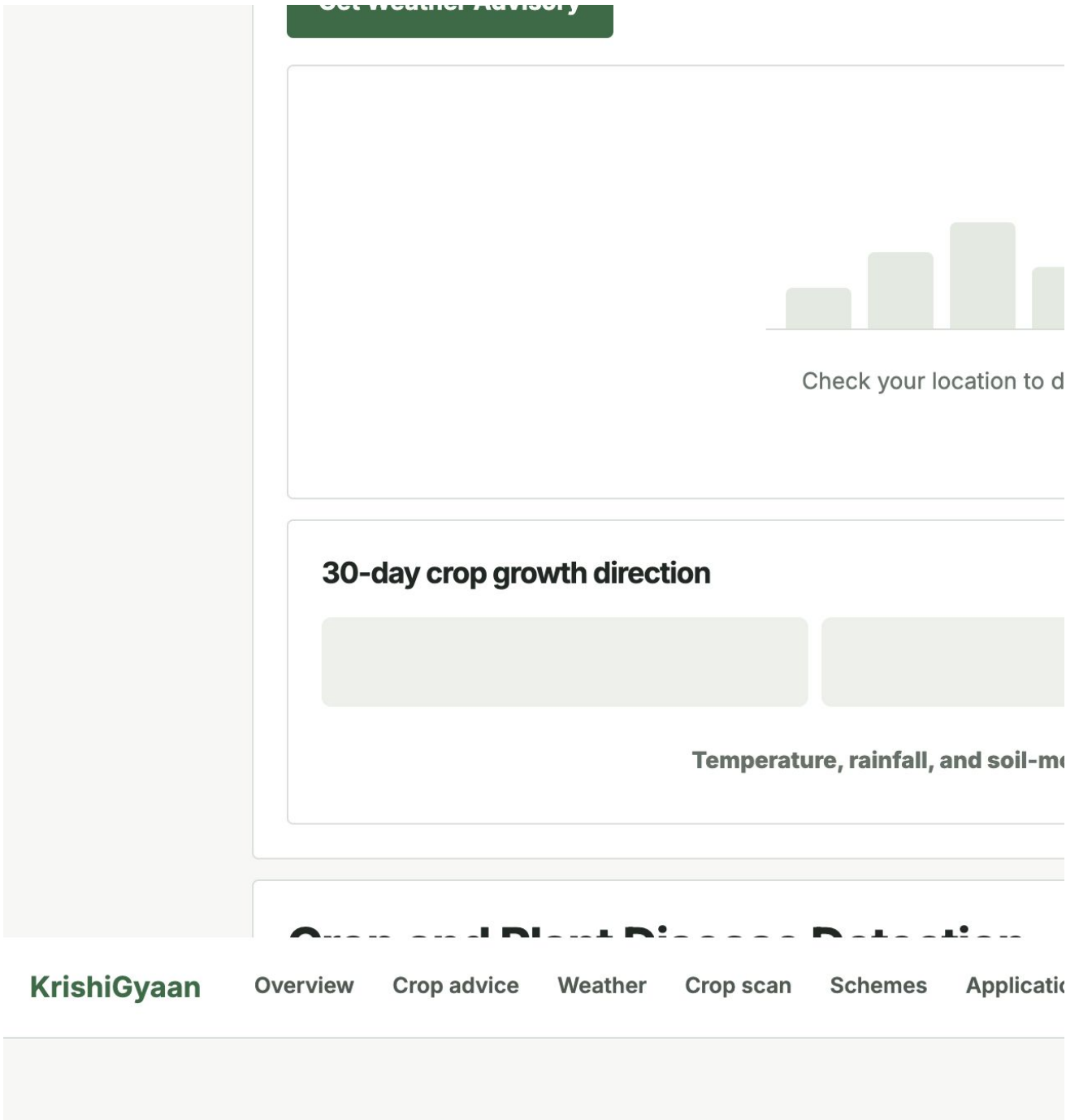
This rendered view documents the implemented interface in reading order. Controls, headings, empty states and data regions are shown as they appear in the running application.



11.4 Farmer Dashboard, segment 1 of 4.

11.4 Farmer Dashboard - View 2 of 4

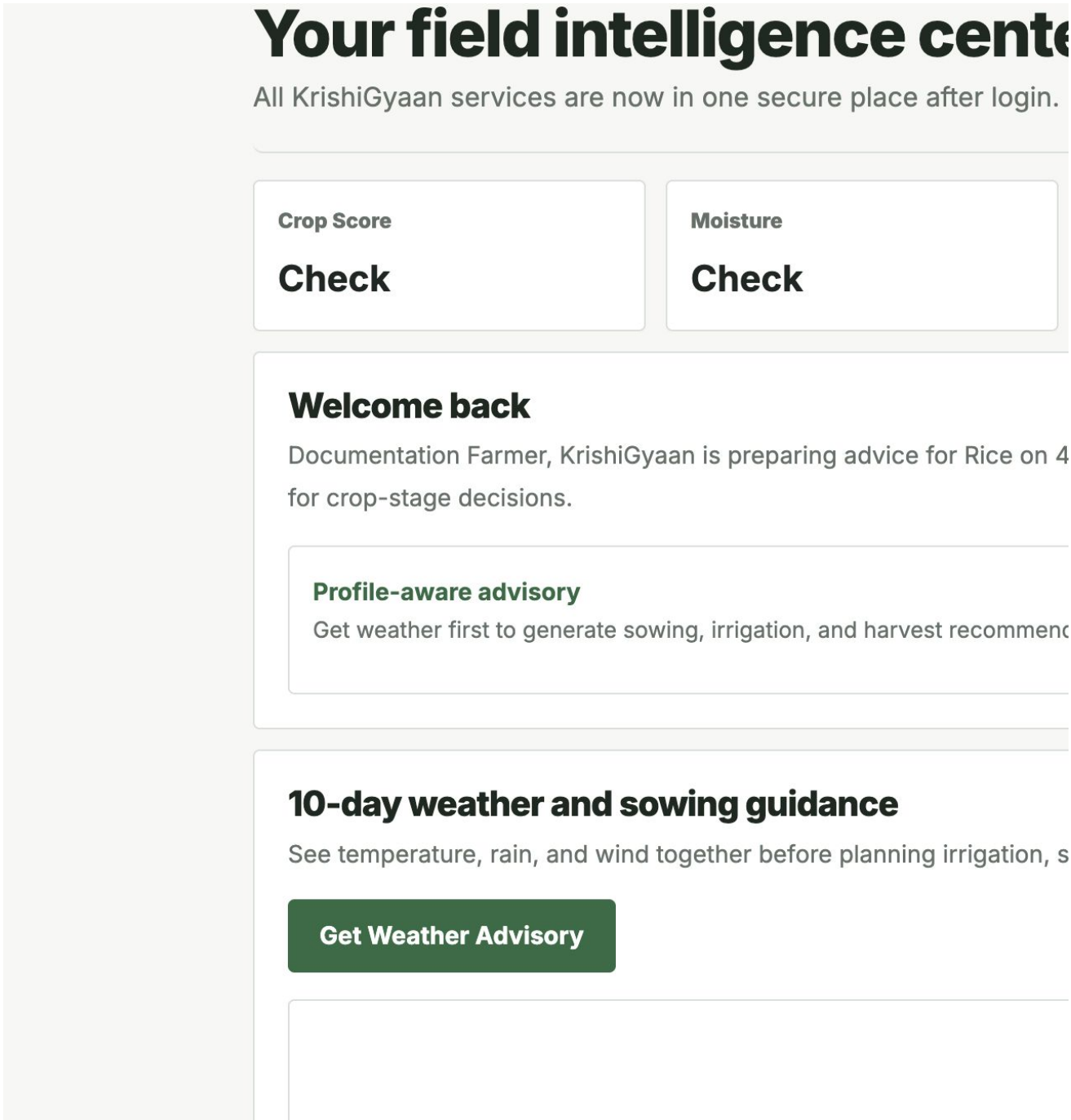
This rendered view documents the implemented interface in reading order. Controls, headings, empty states and data regions are shown as they appear in the running application.



11.4 Farmer Dashboard, segment 2 of 4.

11.4 Farmer Dashboard - View 3 of 4

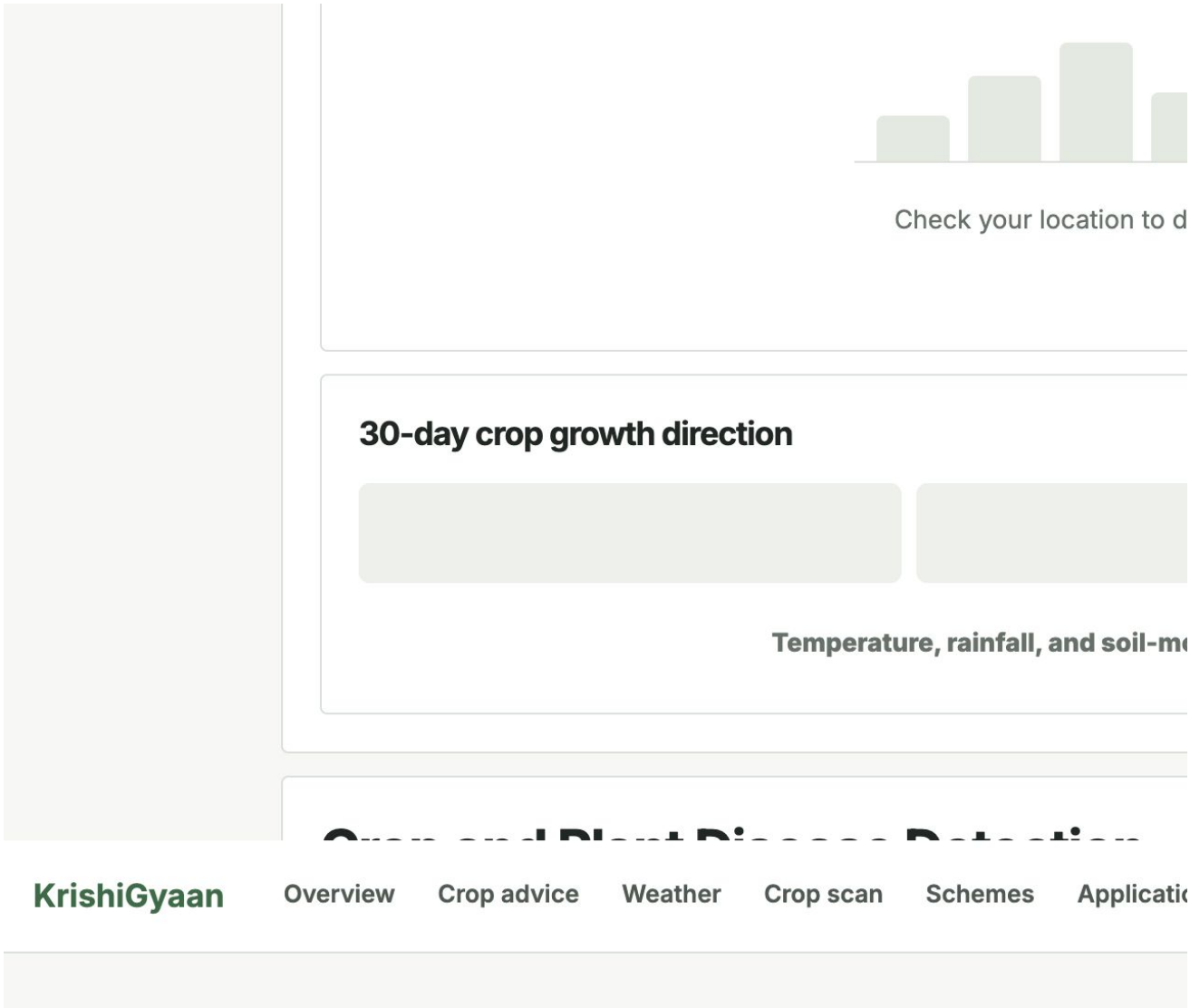
This rendered view documents the implemented interface in reading order. Controls, headings, empty states and data regions are shown as they appear in the running application.



11.4 Farmer Dashboard, segment 3 of 4.

11.4 Farmer Dashboard - View 4 of 4

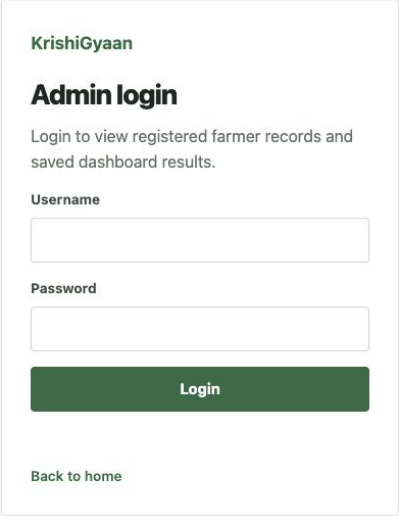
This rendered view documents the implemented interface in reading order. Controls, headings, empty states and data regions are shown as they appear in the running application.



11.4 Farmer Dashboard, segment 4 of 4.

11.5 Administrator Login - View 1 of 1

This rendered view documents the implemented interface in reading order. Controls, headings, empty states and data regions are shown as they appear in the running application.



The screenshot displays the 'Admin login' interface for KrishiGyaan. The page is centered on a light gray background. It features a white rectangular form with a thin gray border. At the top of the form, the text 'KrishiGyaan' is displayed in a dark green font. Below this, the heading 'Admin login' is shown in a bold, black font. A descriptive line of text follows: 'Login to view registered farmer records and saved dashboard results.' The form contains two input fields: 'Username' and 'Password', both with light gray borders. Below the password field is a dark green 'Login' button with white text. At the bottom of the form, there is a link labeled 'Back to home'.

11.5 Administrator Login, segment 1 of 1.

11.6 Administrator Data Centre - View 1 of 3

This rendered view documents the implemented interface in reading order. Controls, headings, empty states and data regions are shown as they appear in the running application.

KrishiGyaan Admin

Farmer records

2 registered farmers

Add farmer

Search

Name or mobile

Documentation Farmer

9998887776

0 images · data saved

Piyush Nath

9123121994

1 images · data saved

Farmer data center

Documentation

Last dashboard update: 3/8/2026, 8:44:11 p

Registered profile

Name

Documentation Farmer

Gender

Male

State

Bihar

Ownership

Owned

Crop

11.6 Administrator Data Centre, segment 1 of 3.

11.6 Administrator Data Centre - View 2 of 3

This rendered view documents the implemented interface in reading order. Controls, headings, empty states and data regions are shown as they appear in the running application.

Rice

Fertilizer
Organic and DAP

Aadhaar last 4
1234

Internet access
Smartphone

Uploaded images

No crop or soil images have been uploaded.

1. 10-day field action plan

Profile-aware advisory

Get weather first to generate sowing, irrig

KrishiGyaan Admin

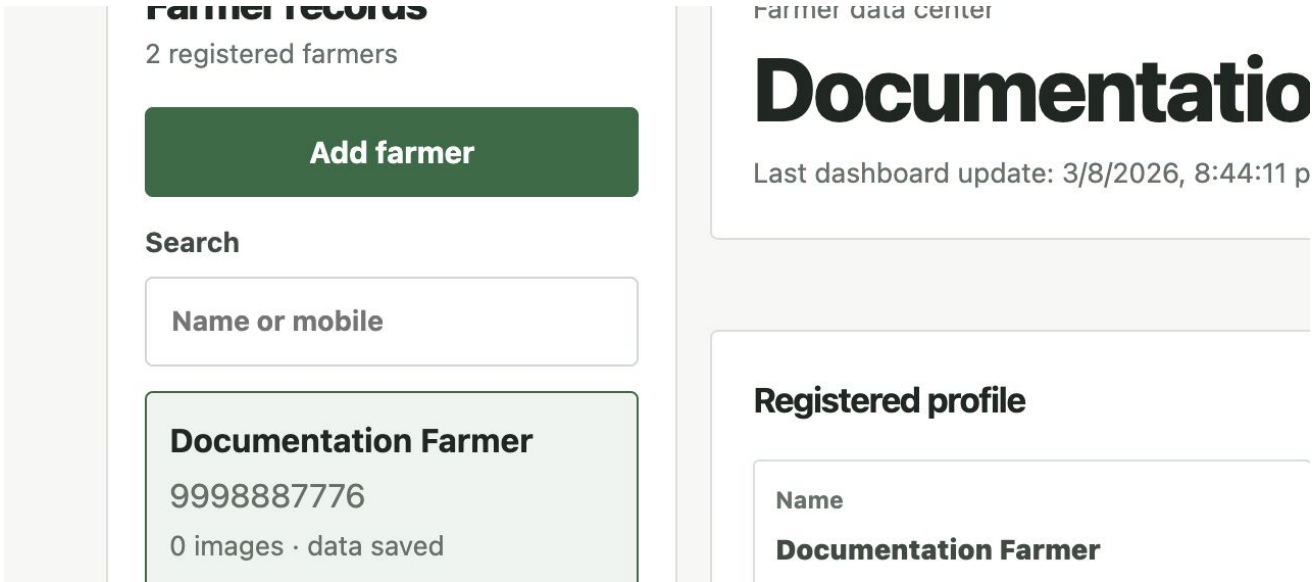
Farmer records

Farmer data center

11.6 Administrator Data Centre, segment 2 of 3.

11.6 Administrator Data Centre - View 3 of 3

This rendered view documents the implemented interface in reading order. Controls, headings, empty states and data regions are shown as they appear in the running application.



11.6 Administrator Data Centre, segment 3 of 3.

12. Output and Report Design

Dashboard results are designed as readable decision-support outputs rather than raw JSON. Weather uses compact visual metrics; disease results separate provider confidence from treatment guidance; drafts preserve printable paragraph structure.

Administrative output groups profile data, uploads and saved results by farmer while keeping editing hidden until requested.

13. Deployment and Operations

Vercel rewrites route static pages to frontend files and preserve the public crop-health and plant-health paths. Production, Preview and Development environments require matching server-side variables.

Operational checks include provider-key validity, MongoDB network access, Cloudinary configuration, route smoke tests and secret rotation.

14. Limitations

The prototype depends on internet services and their quotas. Image diagnosis can be uncertain, weather is probabilistic, AI output can require expert verification, and scheme rules may change.

The forgot-password flow is not production recovery, farmer sessions are browser-based, and the platform does not yet provide formal audit logs, role hierarchy, laboratory soil tests or offline live analysis.

15. Future Scope

Future work can introduce OTP authentication, verified agriculture-officer review, market prices, SMS/WhatsApp delivery, geospatial field records, crop-stage calendars, richer audit logs and automated official-scheme synchronisation.

A progressive offline mode and smaller language models could improve low-connectivity access. Phase 2 documentation will explain the source code module by module.

16. Conclusion

KrishiGyaan demonstrates how a focused web architecture can connect farmer profile data with weather, schemes, image analysis, soil guidance and multilingual AI support.

The completed system satisfies the principal academic objectives of database design, web programming, API integration, security, testing, cloud deployment and structured documentation while remaining honest about advisory limitations.

17. Bibliography and References

- BCA Project Report Guidelines supplied with the project brief.
- Aryabhatta Knowledge University certificate formats supplied with the project brief.
- Node.js Documentation - <https://nodejs.org/docs>
- MongoDB Manual - <https://www.mongodb.com/docs>
- Vercel Documentation - <https://vercel.com/docs>
- Groq API Documentation - <https://console.groq.com/docs>
- Open-Meteo Documentation - <https://open-meteo.com/en/docs>
- Cloudinary Image Upload API - https://cloudinary.com/documentation/image_upload_api_reference
- Crop Kindwise API documentation.
- Plant.id API documentation.
- Pressman and Maxim, Software Engineering: A Practitioner's Approach.
- Sommerville, Software Engineering.